

Final Project: Credit Card Fraud Detection

MLOps

Prepared by: Waberi Waberi

Date: September 27, 2026

1. Executive Summary

This project implements an end-to-end machine learning operations workflow for credit card fraud detection. An XGBoost classifier was developed and evaluated on a highly imbalanced credit-card transaction dataset, packaged behind a FastAPI prediction service, tested automatically, containerized with Docker, orchestrated with Docker Compose, version-controlled with Git/GitHub, and validated through a GitHub Actions CI/CD workflow.

2. Project Objectives

- Develop and evaluate a fraud-detection classifier suitable for imbalanced transaction data.
- Persist the trained model and preprocessing artifacts for repeatable inference.
- Expose predictions through a documented REST API.
- Package the service in Docker and support local orchestration with Docker Compose.
- Implement automated tests and CI/CD validation through GitHub Actions.
- Provide technical documentation, setup instructions, usage guidance, and reproducible evidence.

3. Data and Modeling

The project uses the credit-card fraud dataset containing anonymized V1-V28 features together with Time, Amount, and the binary Class target. The class-distribution visualization demonstrates severe class imbalance: legitimate transactions dominate while fraud cases are rare. This makes accuracy alone potentially misleading, so fraud-class precision, recall, F1-score, ROC-AUC, and confusion-matrix counts were emphasized.

```
plt.title("Credit Card Transaction Class Distribution")
plt.xlabel("Class (0 = Legitimate, 1 = Fraud)")
plt.ylabel("Number of Transactions")
plt.xticks(rotation=0)

plt.tight_layout()
plt.show()
```

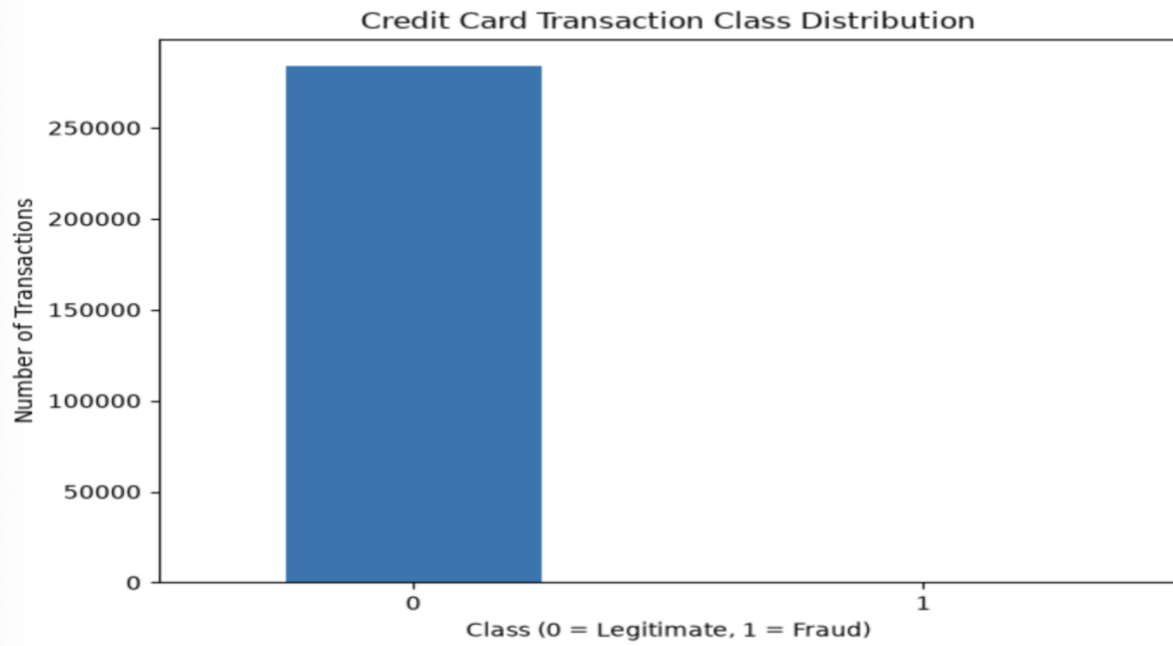


Figure 1. Credit-card transaction class distribution.

Confusion Matrix:
[[42627 20]
[15 59]]

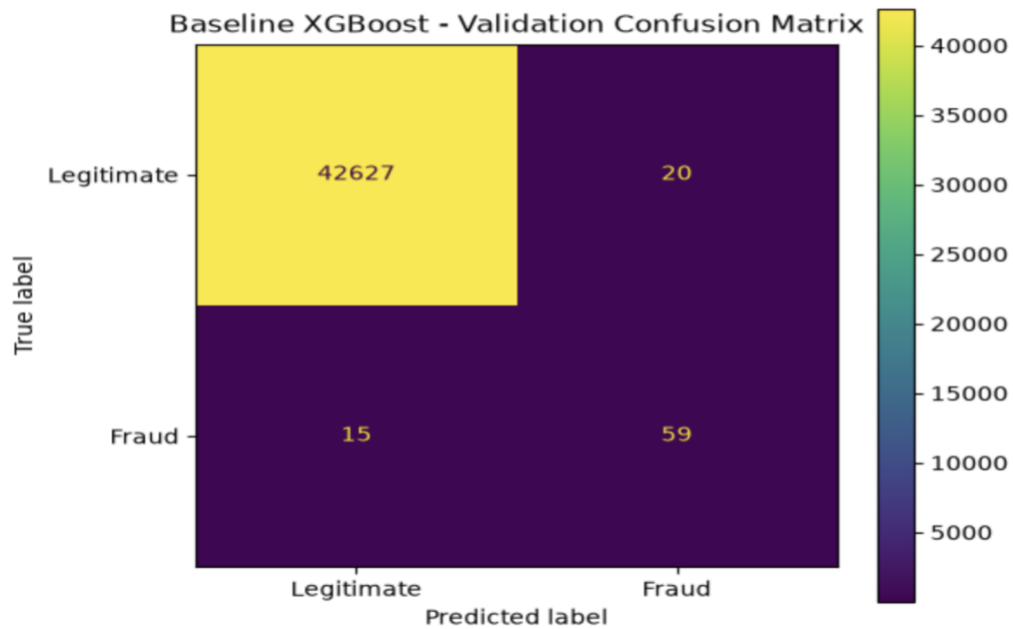


Figure 2. Baseline XGBoost validation confusion matrix.

[24]:

```
# Final evaluation on the untouched test set

y_test_pred = best_model.predict(X_test)
y_test_prob = best_model.predict_proba(X_test)[:, 1]

test_precision = precision_score(y_test, y_test_pred)
test_recall = recall_score(y_test, y_test_pred)
test_f1 = f1_score(y_test, y_test_pred)
test_roc_auc = roc_auc_score(y_test, y_test_prob)

print("Final XGBoost Test Results")
print("-----")
print(f"Precision : {test_precision:.4f}")
print(f"Recall : {test_recall:.4f}")
print(f"F1-Score : {test_f1:.4f}")
print(f"ROC-AUC : {test_roc_auc:.4f}")

print("\nClassification Report:")
print(classification_report(y_test, y_test_pred, digits=4))
```

Final XGBoost Test Results

Precision : 0.8696
Recall : 0.8108
F1-Score : 0.8392
ROC-AUC : 0.9686

Classification Report:

	precision	recall	f1-score	support
0	0.9997	0.9998	0.9997	42648
1	0.8696	0.8108	0.8392	74
accuracy			0.9995	42722
macro avg	0.9346	0.9053	0.9194	42722
weighted avg	0.9994	0.9995	0.9995	42722

Figure 3. Final XGBoost test metrics and classification report.

Final Test Confusion Matrix:
[[42639 9]
[14 60]]

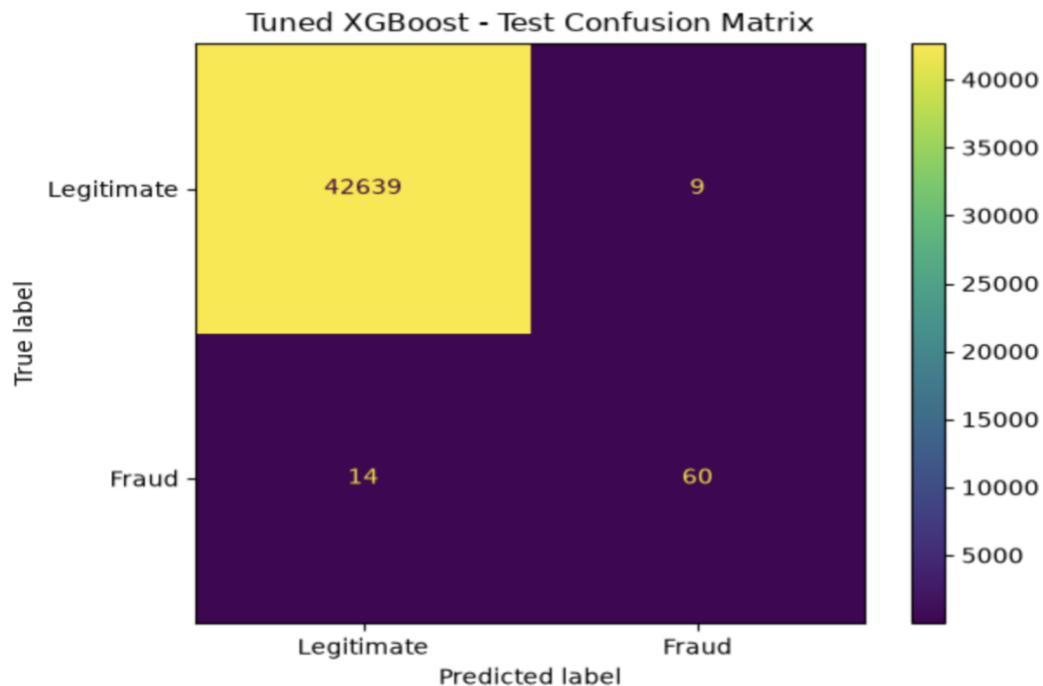


Figure 4. Tuned XGBoost final test confusion matrix.

4. Final Model Evaluation

Metric	Final Test Result
Precision (fraud)	0.8696
Recall (fraud)	0.8108
F1-score (fraud)	0.8392
ROC-AUC	0.9686
Accuracy	0.9995
True Negatives	42,639
False Positives	9
False Negatives	14
True Positives	60

On the untouched test set, the tuned XGBoost model correctly identified 60 of 74 fraud cases and produced 9 false alarms among 42,648 legitimate transactions. Fraud precision of 0.8696 means most transactions flagged as fraud were truly fraudulent, while recall of 0.8108 indicates that approximately 81% of fraud cases in the test set were detected. The ROC-AUC of 0.9686 indicates strong ranking performance across classification thresholds.

5. API Deployment

The trained model is served with FastAPI. At application startup, the service loads the XGBoost model, scaler, and feature metadata. Pydantic validates the 30 required input features. The service preserves training feature order, scales Time and Amount, and returns the predicted class, human-readable label, and fraud probability.

GET / - Confirms that the API is running.

GET /health - Returns health status, model type, and number of expected features.

POST /predict - Accepts transaction features and returns the fraud prediction and probability.

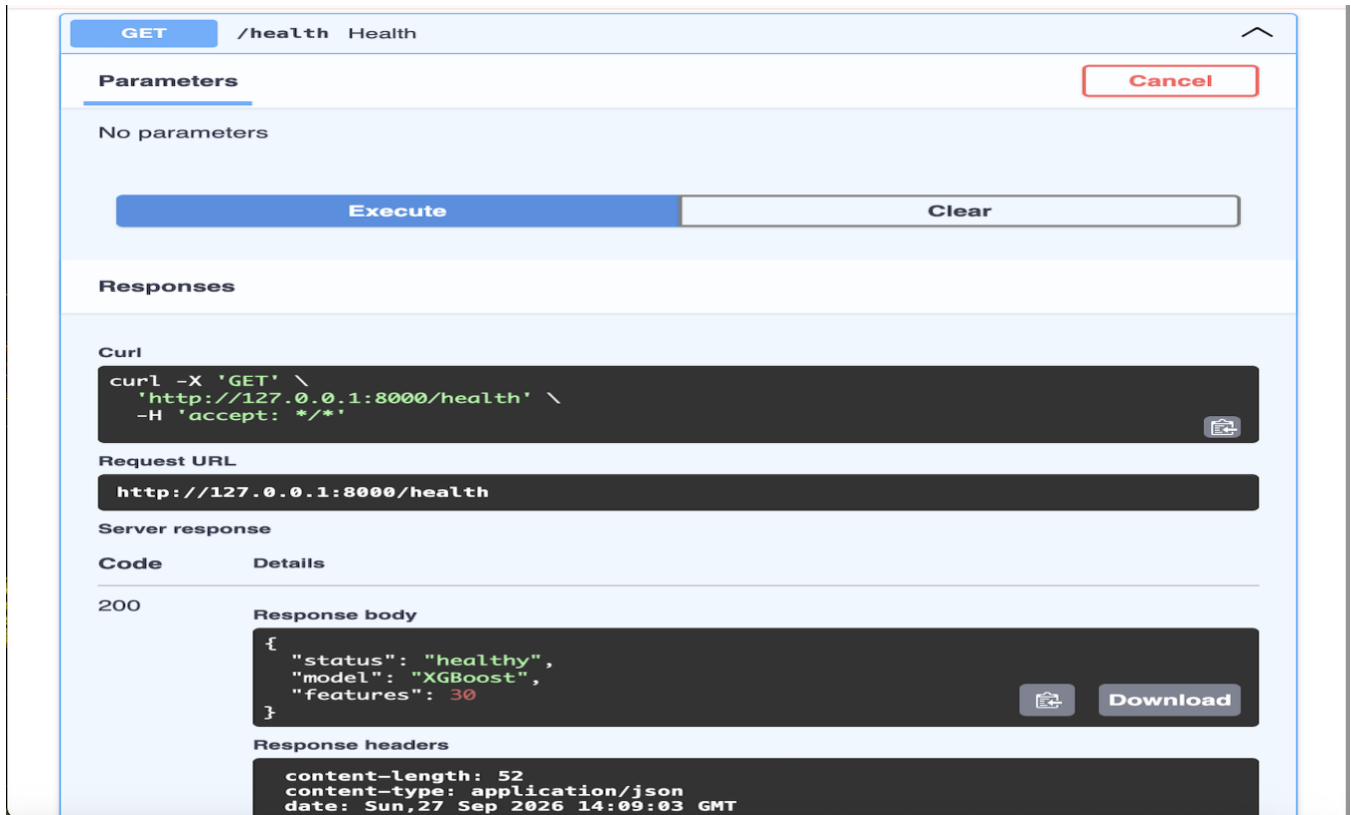
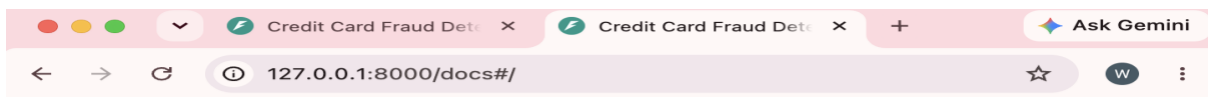


Figure 5. FastAPI health endpoint returning HTTP 200 and healthy status.



Credit Card Fraud Detection API 1.0.0 OAS 3.1

/openapi.json

XGBoost API for detecting potentially fraudulent credit card transactions.

default

GET	/ Root
GET	/health Health
POST	/predict Predict

Schemas

HTTPValidationError > Expand all **object**

Transaction > Expand all **object**

ValidationError > Expand all **object**

Figure 6. Swagger/OpenAPI interface exposing root, health, and prediction endpoints.

6. Containerization and Orchestration

The Dockerfile uses Python 3.11 slim, installs the runtime system dependency libgomp1, installs Python packages from requirements.txt, copies the API and model artifacts, exposes port 8000, and launches Uvicorn. Docker Compose provides a repeatable one-command local deployment and maps host port 8000 to the container.

```
Final Project — -zsh — 80x57
COPY app/ ./app/
COPY models/ ./models/

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
(fraud_mlops) waberiwaberi@Mac Final Project % cat requirements.txt
fastapi
uvicorn[standard]
pandas
numpy
scikit-learn
xgboost
joblib
pydantic
httpx
pytest
(fraud_mlops) waberiwaberi@Mac Final Project % docker build -t fraud-detection-api:1.0 .
[+] Building 74.1s (12/12) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 372B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim 1.3s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                       0.0s
=> [1/7] FROM docker.io/library/python:3.11-slim@sha256:e41613d42d4891e4 0.0s
=> => resolve docker.io/library/python:3.11-slim@sha256:e41613d42d4891e4 0.0s
=> [internal] load build context                                  0.0s
=> => transferring context: 599.10kB                                0.0s
=> CACHED [2/7] WORKDIR /app                                       0.0s
=> [3/7] RUN apt-get update && apt-get install -y libgomp1 && rm 4.8s
=> [4/7] COPY requirements.txt .                                  0.0s
=> [5/7] RUN pip install --no-cache-dir -r requirements.txt       39.7s
=> [6/7] COPY app/ ./app/                                         0.1s
=> [7/7] COPY models/ ./models/                                  0.0s
=> exporting to image                                             28.0s
=> => exporting layers                                              22.3s
=> => exporting manifest sha256:fbdfb7d8221016f50bc522d103e817304d160249 0.0s
=> => exporting config sha256:f01e35041a61cd5fdc5add7b52f5891c63d2641545 0.0s
=> => exporting attestation manifest sha256:ce5cf1f568ceabc3e886d8e459a9 0.0s
=> => exporting manifest list sha256:626d386c481fe673fee418b0ab66f4b0278 0.0s
=> => naming to docker.io/library/fraud-detection-api:1.0         0.0s
=> => unpacking to docker.io/library/fraud-detection-api:1.0     5.7s
(fraud_mlops) waberiwaberi@Mac Final Project % docker ps
CONTAINER ID   IMAGE                                COMMAND                  STATUS    PORTS      NAMES
(fraud_mlops) waberiwaberi@Mac Final Project % docker run -d \
  --name fraud-detection-container \
  -p 8000:8000 \
  fraud-detection-api:1.0
b31a1dcd6047f9afa4f906654825fc72bf4cacf10d8e4b184254a7920ab632ad
(fraud_mlops) waberiwaberi@Mac Final Project % docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED          STATUS      PORTS      NAMES
b31a1dcd6047  fraud-detection-api:1.0             "uvicorn app.main:ap..." 14 seconds ago  Up 14 seconds  0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp  fraud-detection-container
(fraud_mlops) waberiwaberi@Mac Final Project %
```

Figure 7. Successful Docker image build and running fraud-detection container.

7. Automated Testing and CI/CD

The pytest suite validates the root endpoint, health endpoint, and fraud prediction behavior. Local execution completed with 3 tests passed. The GitHub Actions workflow runs on pushes and pull requests to main. It installs Python 3.11 dependencies, executes the tests, then builds the Docker image and validates the Docker Compose configuration.

The submitted GitHub Actions evidence shows the workflow completed successfully in 1 minute 37 seconds, with both 'Test Application' and 'Build Docker Image' jobs passing. The visible Node.js deprecation annotations are warnings from GitHub Action dependencies and did not cause the workflow to fail.

8. Repository Structure and Deliverables

GitHub repository: <https://github.com/welmi496/Final-Project>

- app/ - FastAPI application source
- models/ - XGBoost model, scaler, feature metadata, and metrics
- notebooks/ - fraud detection development notebook
- tests/ - automated API tests
- .github/workflows/ci-cd.yml - CI/CD workflow
- Dockerfile - container build instructions
- docker-compose.yml - orchestration configuration
- requirements.txt - project dependencies
- README.md - overview, setup, testing, Docker, API, and CI/CD instructions
- docs/technical_documentation.md - technical implementation details
- docs/user_guide.md - end-user deployment and API instructions

9. Limitations and Future Improvements

The demonstrated endpoint is accessible locally at 127.0.0.1 rather than through a public cloud URL. For a production deployment, the next improvements would include cloud hosting, TLS, authentication/authorization, rate limiting, centralized logging, data and model-drift monitoring, automated retraining controls, secret management, and alerting. Threshold tuning could also be evaluated against the business cost of false negatives and false positives.

10. Conclusion

The project demonstrates the complete operational path from model development to a tested and containerized inference service with automated CI/CD validation. The tuned XGBoost model achieved fraud precision 0.8696, recall 0.8108, F1-score 0.8392, and ROC-AUC 0.9686 on the untouched test set. The FastAPI service, Docker deployment, Docker Compose configuration, automated tests, Git repository, and successful GitHub Actions run provide reproducible evidence of the MLOps workflow.